

Sieve of Eratosthenes explanation

1. explain sieve of eratosthenis.2
2. explain count sort for c++.5



1. explain sieve of eratosthenis

The **Sieve of Eratosthenes** is a **classic algorithm** used to find **all prime numbers up to a given limit n** efficiently. It works by **eliminating multiples of each prime number**, starting from 2 — the first prime.

Concept

A **prime number** is a number greater than 1 that has **no divisors other than 1 and itself**. Instead of checking each number individually (which is slow), the sieve method **marks all multiples of each prime** as composite (not prime).

Algorithm Steps

Let's say you want to find all primes $\leq n$.

1. Create a boolean array `isPrime[0..n]`, and mark all as `true`. (`true` means "assumed prime" for now)
2. Set `isPrime[0] = false` and `isPrime[1] = false` (since 0 and 1 are not prime).
3. Start from `p = 2`, the first prime.
4. For each number `p` from 2 to $\sqrt{n}$:
 - If `isPrime[p]` is still `true`:
 - Mark all multiples of `p` as `false`:

```
nginx
```

```
for (int i = p*p; i <= n; i += p)
    isPrime[i] = false;
```

- (We start from `p*p` because smaller multiples were already marked earlier.)

5. After the loop, all numbers still marked `true` are **prime numbers**.

Example in C++

cpp

```
#include <iostream>
#include <vector>
using namespace std;

void sieve(int n) {
    vector<bool> isPrime(n + 1, true);

    isPrime[0] = isPrime[1] = false;

    for (int p = 2; p * p <= n; p++) {
        if (isPrime[p]) {
            for (int i = p * p; i <= n; i += p)
                isPrime[i] = false;
        }
    }

    cout << "Prime numbers up to " << n << " are:\n";
    for (int i = 2; i <= n; i++) {
        if (isPrime[i])
            cout << i << " ";
    }
}

int main() {
    int n = 50;
    sieve(n);
    return 0;
}
```

Example Output (for n = 50)

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47

Time and Space Complexity

- **Time Complexity:** $O(n \log \log n)$ → very efficient
- **Space Complexity:** $O(n)$ (to store the boolean array)

Summary

Step	Operation
1	Create a list of numbers up to n
2	Start from 2, mark its multiples
3	Move to the next unmarked number

Step	Operation
4	Repeat until $\sqrt{n}$
5	Remaining unmarked numbers are primes

Would you like me to also show a **visual diagram** or **step-by-step illustration** of how numbers get crossed out (for small n , like 30)?



2. explain count sort for c++

Sure! Let's go step-by-step through **Counting Sort**, one of the most efficient **non-comparison-based sorting algorithms** — great for sorting integers when the range of values is known and not too large.

What is Counting Sort?

Counting Sort works by **counting the number of occurrences** of each unique value in the input array, then **using those counts to place each element directly in its correct sorted position**.

Unlike Bubble Sort or Quick Sort, it doesn't compare elements — it uses **frequency counting**.

Algorithm Steps

Let's say we have an array of integers:

```
ini  
  
arr = [4, 2, 2, 8, 3, 3, 1]
```

Step 1: Find the maximum value

We find the **maximum element (max)** in the array.

→ Here, **max** = 8.

Step 2: Create a count array

Create an array **count** of size **max + 1**, initialized to 0.
count[i] will store how many times **i** appears in **arr**.

Step 3: Count each element

For every element **x** in **arr**, do:

```
CSS
```

```
count[x]++
```

After counting, **count** looks like this:

```
CSS
```

```
Index (i): 0 1 2 3 4 5 6 7 8
Count[i]: 0 1 2 2 1 0 0 0 1
```

Step 4: Compute cumulative sum

This helps us find the **position** of each element in the output array.

```
arduino
```

```
for (int i = 1; i <= max; i++)
    count[i] += count[i - 1];
```

Now **count** represents **positions**:

```
CSS
```

```
Index (i): 0 1 2 3 4 5 6 7 8
Count[i]: 0 1 3 5 6 6 6 6 7
```

Step 5: Build the output array

Now, iterate **backward** through **arr** (to make the sort **stable**) and place each element:

```
lua
```

```
output[count[arr[i]] - 1] = arr[i];
count[arr[i]]--;
```

Step 6: Copy the output back

Copy sorted elements from **output** to **arr**.

Result:

```
ini
```

```
arr = [1, 2, 2, 3, 3, 4, 8]
```



C++ Implementation

cpp

```

#include <iostream>
#include <vector>
using namespace std;

void countingSort(vector<int> &arr) {
    int n = arr.size();

    // Step 1: Find max value
    int maxVal = *max_element(arr.begin(), arr.end());

    // Step 2: Initialize count array
    vector<int> count(maxVal + 1, 0);

    // Step 3: Count occurrences
    for (int num : arr)
        count[num]++;

    // Step 4: Cumulative sum
    for (int i = 1; i <= maxVal; i++)
        count[i] += count[i - 1];

    // Step 5: Build output array (stable)
    vector<int> output(n);
    for (int i = n - 1; i >= 0; i--) {
        output[count[arr[i]] - 1] = arr[i];
        count[arr[i]]--;
    }

    // Step 6: Copy output to original array
    arr = output;
}

int main() {
    vector<int> arr = {4, 2, 2, 8, 3, 3, 1};
    countingSort(arr);

    cout << "Sorted array: ";
    for (int num : arr)
        cout << num << " ";
    return 0;
}

```

Output:

php

Sorted array: 1 2 2 3 3 4 8

Time and Space Complexity

Complexity Type	Description	Value
Time	Counting + Building output	$O(n + k)$
Space	Count + Output arrays	$O(n + k)$
where n = number of elements, k = range (max value)		

When to Use Counting Sort

 When:

- You are sorting **integers (or small positive numbers)**
- The range of numbers (k) isn't too large compared to n

 Avoid when:

- The numbers are **very large** or **negative** (unless adjusted)
- You need to sort **strings or floats** directly

Would you like me to show how to modify Counting Sort to handle **negative numbers** too?